

Programación Avanzada

Ejercicios.

Determinar la precondition para que las siguientes ternas sean correctas:

a) $\{ P \} j := i + 1 \{ j > 0 \}$

$$\{ P \} = \{ Q_E^V \} = \{ V = E, Q \} = \{ (j > 0)_{i+1}^j \} = \{ j = i + 1, \{ j > 0 \} \}$$

$$= \{ i + 1 > 0 \} = \{ i > -1 \}$$

b) $\{ P \} y := x^2 \{ y > 1 \}$

$$\{ P \} = \{ Q_E^V \} = \{ V = E, Q \} = \{ (y > 1)^{x^2}_y \} = \{ y = x^2, \{ y > 1 \} \}$$

$$= \{ x^2 > 1 \} = \{ \sqrt{x^2} > \sqrt{1} \} = \{ |x| > 1 \}$$

$$\begin{array}{c} \leftarrow \quad) \quad (\quad \rightarrow \\ -1 \quad 0 \quad 1 \end{array}$$

Programación Avanzada

Determinar la postcondición para que las siguientes ternas sean correctas:

$$c) \{ x > 2 \} x := x^2 \{ Q \}$$

$$\{ Q \} = \{ C_{\{P\}}^E \} = \{ E = \{ P \}, C \} = \{ (x = x^2)_{\{x>2\}}^{x^2} \}$$

$$= \{ x^2 = \{ x > 2 \}, x := x^2 \}$$

$$\text{pero } P = \{ x > 2 \} \Rightarrow P = \{ x^2 > 4 \}$$

$$\therefore \{ Q \} = \{ (x = x^2)_{\{x^2>4\}}^{x^2} \} \\ = \{ x > 4 \}$$

Recordar que es sustituir en $x := x^2$ el valor de x en $\{ P \}$

$$d) \{ P \} x := 1/x \{ x \geq 0 \}$$

$$\{ P \} = \{ Q_E^V \} = \{ V = E, Q \} = \{ (x \geq 0)_{1/x}^x \} = \{ 1/x \geq 0 \}$$

$$\text{Si } x \neq 0 \text{ entonces } \{ P \} = \{ x^2 / x \geq x^2 0 \} = \{ x > 0 \}$$

Programación Avanzada

Los ejemplos a), b) y d) ejemplifican técnicas que se utilizarán más adelante para demostrar que un código es correcto.

Y el ejemplo c) muestra lo que interesa en la programación: dado un conjunto de datos iniciales y de instrucciones, saber que sigue después de ejecutar estas instrucciones, o saber cual es el resultado después de ejecutarlas. Saber que hace el programa.

Concatenación de código.

La concatenación de código significa que las instrucciones de un programa se ejecutan secuencialmente, de tal forma que el estado final de la primera instrucción de un código, se convierte en el estado inicial de la segunda instrucción del programa y así sucesivamente.

Programación Avanzada

Regla de la concatenación.

Sean $C1$ y $C2$ dos partes de un código y sea $C1; C2$ su concatenación. Si $\{P\}C1\{R\}$ y $\{R\}C2\{Q\}$ son ternas de Hoare correctas, entonces se puede afirmar que:

$$\frac{\begin{array}{c} \{P\}C1\{R\} \\ \{R\}C2\{Q\} \end{array}}{\{P\}C1;C2\{Q\} \dots \text{es correcto}}$$

Ejemplo.

Demostrar que el siguiente código es correcto:

$\{ \} c := a + b; c := c / 2 \{ c = (a + b) / 2 \}$

Hint. Utilizar la regla de la concatenación y de asignación.
Ver diagrama anterior para conocer cómo debe hacerse la demostración.

Programación Avanzada

Encontrar una precondition para la segunda instrucción:

$$\{P\} c := c/2 \{c = (a+b)/2\}$$

$$P = \{Q_E^V\} = \{ (c = (a+b)/2)_{c/2}^c \} = \{ c/2 = (a+b)/2 \} \\ = \{ c = (a+b) \}$$

$$\therefore \{c = (a+b)\} c := c/2 \{c = (a+b)/2\} \dots (1)$$

Encontrar ahora una preconición para la primera instrucción:

$$\{P\} c := a+b \{c = (a+b)\}$$

$$P = \{Q_E^V\} = \{ (c = (a+b))_{a+b}^c \} = \{ (a+b) = (a+b) \} = \{ \}$$

$$\therefore \{ \} c := a+b \{c = (a+b)\} \dots (2)$$

Y aplicando la regla de la concatenación a (1) y a (2) se obtiene que el código es correcto. *Ver solución*

Programación Avanzada

Ejemplo.

Demostrar que el siguiente código es correcto:

$$\{ \} s := 1; s := s + r; s := s + r \times r \{ s = 1 + r + r^2 \}$$

La demostración es muy similar a la anterior, la técnica es a partir de la última instrucción encontrar $\{P\}$ y de ahí sucesivamente al inicio.

$$\{P\} s := s + r \times r \{ s = 1 + r + r^2 \}$$

$$P = \{Q_E^V\} = \{ (s = 1 + r + r^2)_{S+r \times r}^S \} = \{ s + r \times r = 1 + r + r^2 \}$$

$$\therefore \{ s = 1 + r \} s := s + r \times r \{ s = 1 + r + r^2 \} \dots (1)$$

$$\{P\} s := s + r \{ s = 1 + r \}$$

$$P = \{Q_E^V\} = \{ (s = 1 + r)_{S+r}^S \} = \{ s + r = 1 + r \}$$

$$\therefore \{ s = 1 \} s := s + r \{ s = 1 + r \} \dots (2)$$

Programación Avanzada

$$\{P\} s := 1 \{s=1\}$$

$$P = \{Q_E^V\} = \{(s=1)_1^S\} = \{1=1\} = \{\}$$

$$\therefore \{\} s := 1 \{s=1\} \dots (3)$$

Y aplicando la regla de la concatenación a (1), (2) y a (3) se obtiene que el código es correcto. *Ver solución*

Demostrar que el siguiente código es correcto:

$$\{a=A, b=B\} h := a; a := b; b := h \{a=B, b=A\}$$

La demostración es muy similar a la anterior, la técnica es a partir de la última instrucción encontrar $\{P\}$ y de ahí sucesivamente al inicio.

Programación Avanzada

$$\{P\} b := h \{a = B, b = A\}$$

$$P = \{Q_E^V\} = \{(a = B, b = A)_{h}^b\} = \{a = B, h = A\}$$

$$\therefore \{a = B, h = A\} b := h \{a = B, b = A\} \dots (1)$$

$$\{P\} a := b \{a = B, h = A\}$$

$$P = \{Q_E^V\} = \{(a = B, h = A)_{b}^a\} = \{b = B, h = A\}$$

$$\therefore \{b = B, h = A\} a := b \{a = B, h = A\} \dots (2)$$

$$\{P\} h := a \{h = A, b = B\}$$

$$P = \{Q_E^V\} = \{(h = A, b = B)_{a}^h\} = \{a = A, b = B\}$$

$$\therefore \{a = A, b = B\} h := a \{h = A, b = B\} \dots (3)$$

Y nuevamente aplicando la regla de la concatenación a (1), (2) y a (3) se obtiene que el código es correcto. *Ver solución*

Programación Avanzada

Invariante de un código

Cualquier aserción que es a la vez precondition y postcondición de un código se denomina invariante. Significa que el código no afecta a la aseveración que se hace después de ejecutar el código. La relación de variables se mantiene antes y después de ejecutar el código.

Ejemplo.

Demostrar que $r = 2^i$ es un invariante del siguiente código:

$i := i + 1; r := r \times 2.$

Es decir, demostrar que la siguiente terna de Hoare es correcta:

$$\{ r = 2^i \} i := i + 1; r := r \times 2 \{ r = 2^i \}$$

Programación Avanzada

$$\{ P \} r := r \times 2 \{ r = 2^i \}$$

$$P = \{ Q_E^V \} = \{ (r = 2^i)_r^r \times 2 \} = \{ r \times 2 = 2^i \}$$

$$\therefore \{ r = 2^{i-1} \} r := r \times 2 \{ r = 2^i \} \dots (1)$$

$$\{ P \} i := i + 1 \{ r = 2^{i-1} \}$$

$$P = \{ Q_E^V \} = \{ (r = 2^{i-1})_{i+1}^i \} = \{ r = 2^{i+1-1} \} = \{ r = 2^i \}$$

$$\therefore \{ r = 2^i \} i := i + 1 \{ r = 2^{i-1} \} \dots (2)$$

Y aplicando la regla de la concatenación a (1) y a (2) se obtiene que el código es correcto. *Ver solución.*

Programación Avanzada

El invariante en este ejemplo, se puede ver intuitivamente de la siguiente forma. Por ejemplo:

Si $i = 2$:

$\{ r = 2^2 \} i := 2 + 1; r := 2^2 \times 2 \{ r = 2^3 \}$ sustituyendo el valor de i y de r



$2^3 \therefore$ sí se cumple la postcondición $r = 2^3$

Si $i = 6$:

$\{ r = 2^6 \} i := 6 + 1; r := 2^6 \times 2 \{ r = 2^7 \}$ sustituyendo el valor de i y de r



$2^7 \therefore$ sí se cumple la postcondición $r = 2^7$

Programación Avanzada

Y algebraicamente:

$\{ r = 2^i \} i := i + 1; r := 2^i \times 2 \{ r = 2^{i+1} \}$ sustituyendo el valor de i y de r



$2^{i+1} \therefore$ sí se cumple la postcondición $r = 2^{i+1}$

Por otra parte, es cierta la relación:

$\wr \{ r = 2^i \} i := i + 1 \{ r = 2^{i-1} \}$? Que es la relación (2) que se obtuvo en la demostración anterior. *Ver solución*

Programación Avanzada

La regla de inferencia para la sentencia if sin cláusula else

Si $C1$ es una parte de un programa y B es una condición, entonces la sentencia '*if B then C1*' se interpreta de la siguiente forma: *si B es verdadero se ejecuta C1*.

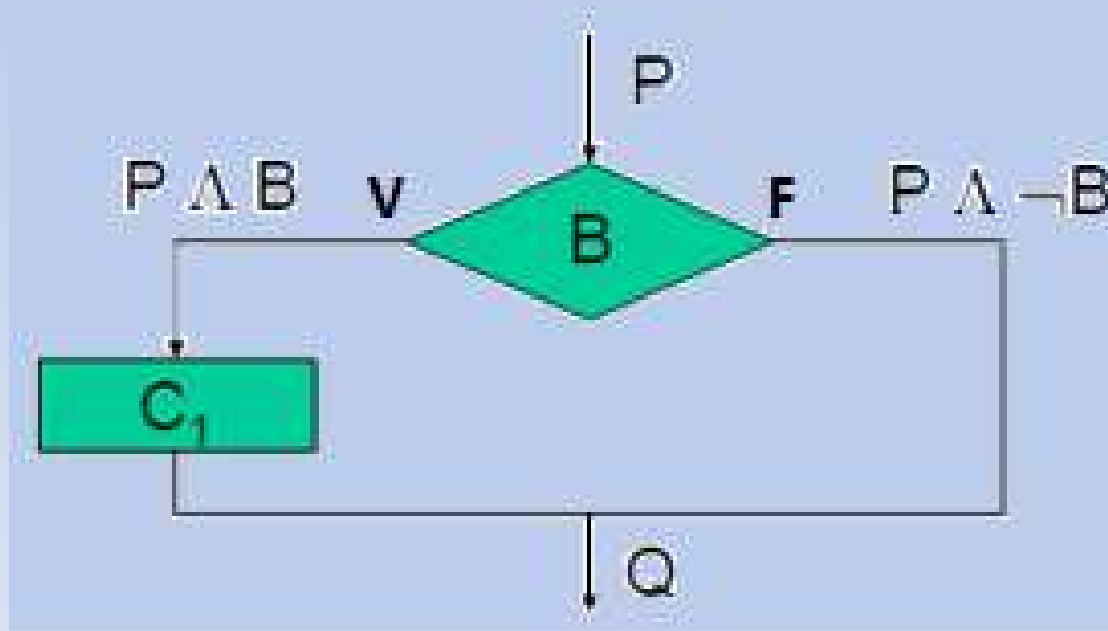
Si ahora se desea demostrar la corrección de una sentencia 'if' con una precondition $\{P\}$ y una postcondition $\{Q\}$ tendremos dos posibilidades:

- Si el estado inicial satisface B además de P , entonces se ejecutará $C1$ y por tanto la verificación equivaldrá a demostrar que $\{P \wedge B\} C1 \{Q\}$ es correcto.

- Si el estado inicial no satisface B , entonces esto equivaldrá a demostrar que $\{P \wedge \neg B\} \Rightarrow \{Q\}$.

Programación Avanzada

$$\frac{\begin{array}{l} \{P \wedge B\} C_1 \{Q\} \\ \{P \wedge \neg B\} \Rightarrow \{Q\} \end{array}}{\{P\} \text{if } B \text{ then } C_1 \{Q\}}$$



Programación Avanzada

Ejemplo: Demostrar que es cierta la siguiente terna de Hoare:

$$\{ \} \text{ if } (\text{max} < a) \text{ then } \text{max} := a \{ \text{max} \geq a \}$$

Intuitivamente.

Si $\text{max} < a \Rightarrow \text{max} := a \{ \text{max} = a \}$

Si $\neg(\text{max} < a) \Rightarrow \text{max} \geq a \{ \text{max} \geq a \}$

} $\{ \text{max} \geq a \}$

Demostración.

Sup. $\neg(\text{max} < a)$.

$\neg(\text{max} < a) \Rightarrow \text{max} \geq a$ Pero $\neg(\text{max} < a) = \{ \{ \} \wedge \neg(\text{max} < a) \}$

$\therefore \{ \{ \} \wedge \neg(\text{max} < a) \} \Rightarrow \{ \text{max} \geq a \} \dots (1)$

Sup. $(\text{max} < a)$.

P. D. $\{ \text{max} < a \} \text{max} := a \{ \text{max} \geq a \}$

Programación Avanzada

$$P = \{Q_E^V\} = \{(\max \geq a)_a^{\max}\} = \{a \geq a\}$$

$$\therefore \{a \geq a\} \max := a \{ \max \geq a \} \dots (2)$$

Por otra parte:

$\{a = a\} \Rightarrow \{a \geq a\}$ entonces:

$$\begin{array}{c} \{a = a\} \Rightarrow \{a \geq a\} \\ \{a \geq a\} \max := a \{ \max \geq a \} \dots \text{es (2)} \\ \hline \{a = a\} \max := a \{ \max \geq a \} \end{array}$$

y como $\{a = a\} = \{\}$ entonces:

$$\{\} \max := a \{ \max \geq a \} \dots (3)$$

Y como $\{\max < a\} \Rightarrow \{\}$, porque cualquier aseveración fortalece la vacía, entonces:

Programación Avanzada

$$\frac{\begin{array}{l} \{ \max < a \} \Rightarrow \{ \} \\ \{ \} \max := a \{ \max \geq a \} \dots \text{es (3)} \end{array}}{\{ \max < a \} \max := a \{ \max \geq a \}}$$

Es decir:

$$\{ \{ \} \wedge \{ \max < a \} \max := a \{ \max \geq a \} \dots \text{(4)}$$

Finalmente aplicando la regla de inferencia para la sentencia if sin cláusula else:

$$\frac{\begin{array}{l} \{ \{ \} \wedge \{ \max < a \} \max := a \{ \max \geq a \} \dots \text{es 4} \\ \{ \{ \} \wedge \neg(\max < a) \} \Rightarrow \{ \max \geq a \} \dots \text{es (1)} \end{array}}{\{ \} \text{if } (\max < a) \text{ then } \max := a \{ \max \geq a \}}$$

Programación Avanzada

La regla de inferencia para la sentencia if con cláusula else.

Si $C1$ y $C2$ son dos partes de un programa y si B es una condición, entonces la sentencia '*if B then $C1$ else $C2$* ' se interpreta de la siguiente forma: *si B es verdadero se ejecuta $C1$ sino $C2$.*

Si ahora se desea demostrar la corrección de una sentencia 'if' con una precondition $\{P\}$ y una postcondition $\{Q\}$ tendremos dos posibilidades:

- Si el estado inicial satisface B además de P entonces se ejecutará $C1$ y por tanto la verificación equivaldrá a demostrar que $\{P \wedge B\} C1 \{Q\}$ es correcto.

- Si el estado inicial no satisface B entonces se ejecutará $C2$ y por tanto la verificación equivaldrá a demostrar que $\{P \wedge \neg B\} C2 \{Q\}$ es correcto.

Programación Avanzada

$$\frac{\begin{array}{l} \{P \wedge B\} C_1 \{Q\} \\ \{P \wedge \neg B\} C_2 \{Q\} \end{array}}{\{P\} \text{ if } B \text{ then } C_1 \text{ else } C_2 \{Q\}}$$
